PagedAttention in MiniTorch:

KV Cache Efficiency, Prefix Reuse, and Memory Sharing

NingRui Yang Yixiang Dai
11868 Large Language Model Systems
Carnegie Mellon University

ONE-SENTENCE SUMMARY

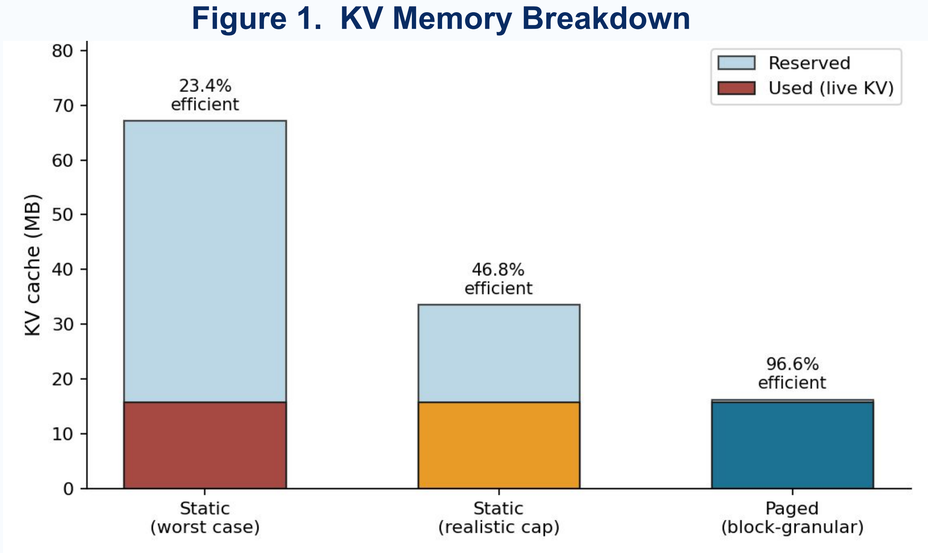
We build a paged KV-cache runtime in MiniTorch, improving memory efficiency and enabling longer-context, faster inference under fixed memory budgets.

MOTIVATION

KV cache often bottlenecks LLM serving.
Traditional contiguous allocation reserves worst-case space, wasting memory when requests are shorter than the maximum context.

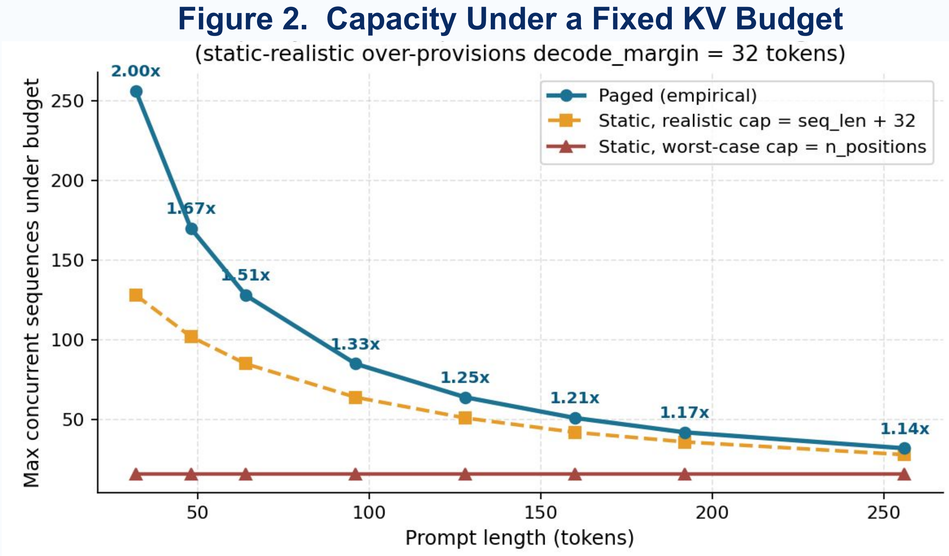
RESULTS

Figure 1. KV Memory Breakdown



Paged allocation keeps a much larger fraction of reserved KV memory useful.

Figure 2. Capacity Under a Fixed KV Budget



Paged allocation supports more concurrent sequences under the same KV budget, especially as prompts get longer.

Figure 3. Parallel Sampling KV Memory (KV blocks used)

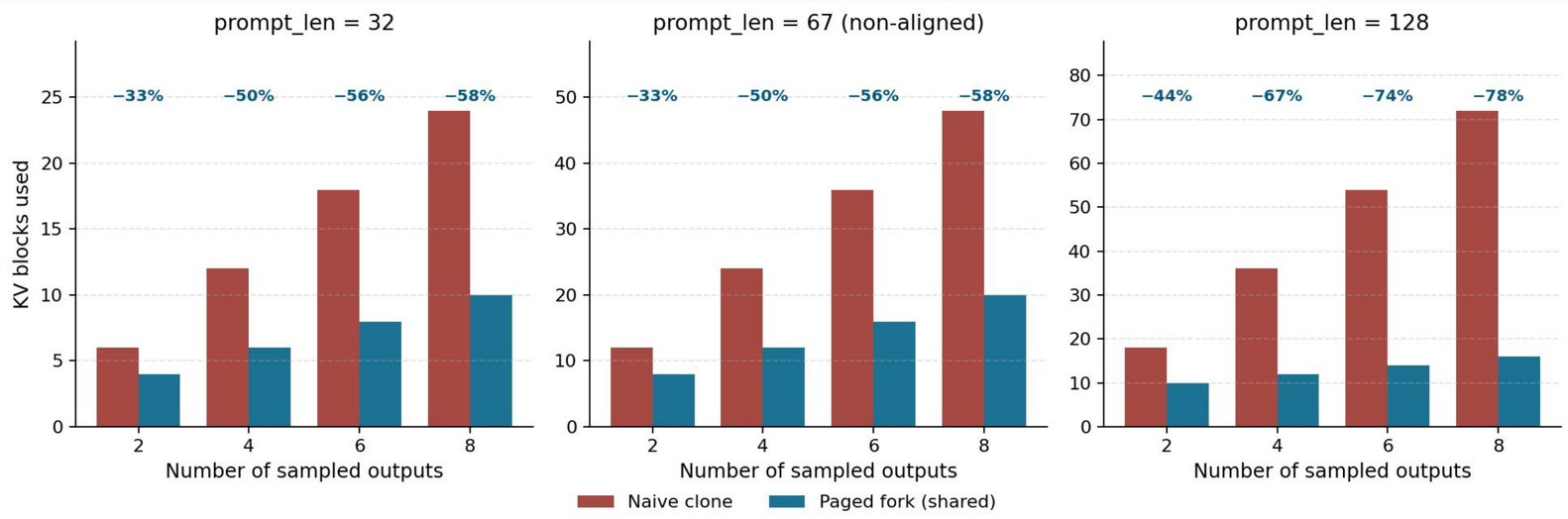
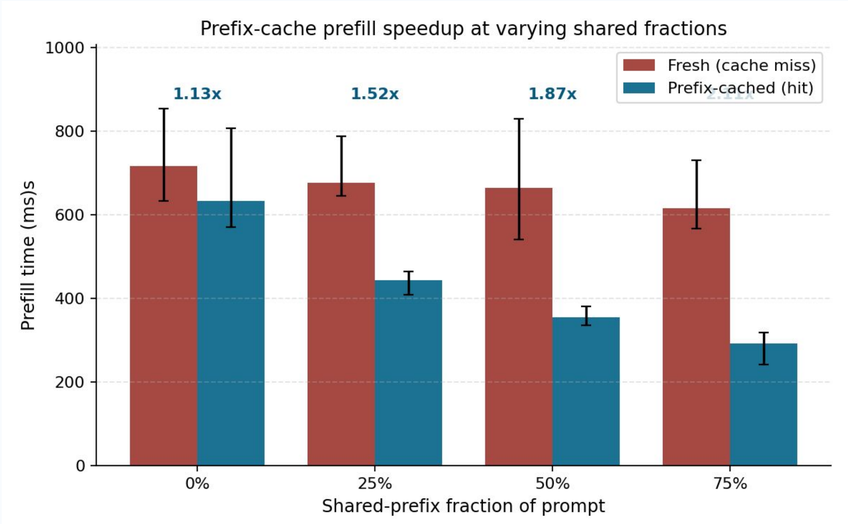


Figure 4. Prefix-Cache Prefill Speedup



Paged KV cache
KV states are stored in fixed-size blocks.

Block tables
Each sequence maps logical token blocks to physical blocks.

Prefix caching
Reuse full prompt blocks across requests.

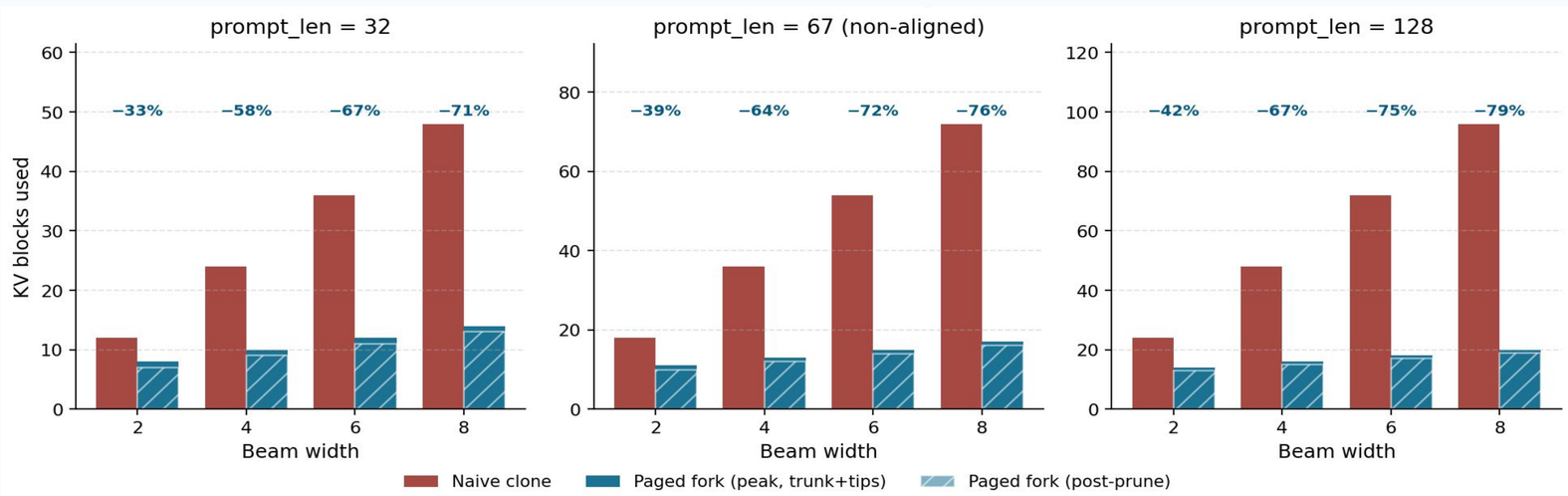
Runtime residency
KV stays on device across decode steps.

Fork + copy-on-write
Outputs share prefix blocks until they diverge.

PROBLEM STATEMENT

1. How much KV memory does paging save?
2. How much capacity does paging add under a fixed budget?
3. How much does prefix caching speed up prefill?
4. How much memory does parallel sampling save?
5. How much memory does beam search save?

Figure 5. Beam Search KV Memory (KV blocks used)



TAKEAWAY

Block-based KV management improves memory efficiency and enables scalable decoding.